

01.10.2026 · 13:00 · 90 минут



ПРАКТИКУМ «ИИ В РАБОТЕ АНАЛИТИКА» · ВЕБИНАР 1

ИИ в работе аналитика: подход и рабочий цикл

Алёна Киселёва · Николай Карпов · Женя Бадмаева



Программа

0–7	1. Зачем мы это делаем	
7–20	2. Что умеет ИИ и от чего зависит качество	
20–23	Вопросы по блоку 2	
23–40	3. Как устроено рабочее место	
40–52	4. Универсальный рабочий цикл	
52–62	5.1. Кейс: ДБО ФЛ ЦР ГенБанк	
62–70	5.2. Кейс: ДБО ЮЛ Универсал Банк (Узбекистан)	
70–79	6. Как начать у себя	
79–87	7. Вопросы и ответы	
87–90	Обратная связь	





БЛОК 1

0–7 минут





ИИ в производственном процессе

Цели

Встроить ИИ в повседневную работу аналитиков: разбирать источники и готовить материалы по общему подходу.

Польза для аналитиков

Навыки, над которыми мы сейчас работаем, по значению сравнимы с приходом компьютеров и интернета. Кто их проигнорировал — выпал из рынка. Кто освоил — ушёл далеко вперёд конкурентов.

Польза для компании

Сегодня опыт аналитика живёт у него в голове и дальше не расходится. Мы настраиваем горизонтальное распространение опыта: то, чего добился один, работает у всех.

Рефлексия в конце дня переводит этот опыт в записи: ошибка → решение → правило. Каждый следующий аналитик и каждая следующая сессия агента стартуют не с нуля, а с накопленного.

На этих записях строится корпоративный ИИ, который знает наш контекст, наши документы и наши грабли — вместо покупных синтетических данных.

Сроки и ожидания

Сегодня — показ. На этой неделе — разворачиваете у себя по навигатору. К следующей встрече — первая задача выполнена, результат и время записаны.



Число в чат. Если чат молчит — не вытягиваем, идём дальше.





БЛОК 2

7–20 минут





С чем вы уйдёте

После вебинара каждый сможет **развернуть у себя типовое окружение**, положить туда источники своего проекта и **выполнить первую задачу по стандартному алгоритму**.

Сегодня показываем, как это устроено. Ставить будете у себя после встречи.

Чего сегодня не будет

готового ТЗ по одной кнопке · цифр ускорения, выданных за замер · настройки у каждого в прямом эфире





Что умеет инструмент

Забирает механическую часть

читает разрешённые файлы, строит реестры и сравнения, готовит черновики

Решения остаются за вами

аналитические решения и приёмку по-прежнему принимает аналитик

Схемы и скриншоты — тоже источник: в пакете есть модели, которые переводят картинки в машиночитаемый вид.

Уже работает на наших проектах:

- Регуляtorика из PDF — в машиночитаемый вид
- Сверка РП BC, XSD и ТЗ-Форматы: расхождения отдельно от вопросов
- Требования к интерфейсу по макетам Figma и БФТ, журнал расхождений по методам
- UML-диаграммы в формате PlantUML по готовым требованиям — агент строит сам



Ограничения и риски

- **У модели нет кнопки «я не знаю».** Правдоподобно — не значит верно. Нет данных — вопрос, а не заполненный раздел. Каждый факт проверяем по источнику.
- **Модель видит только то, что ей дали сейчас.** Что не попало в контекст — того для неё нет.
- **Документы клиентов, персональные данные, банковская тайна, данные промышленного контура** — в модель не передаются без отдельного разрешения.
- **Модели под корпоративным VPN:** китайские модели пакета Alibaba работают, включая Kimi. GPT и Claude — нет.
- **Git — только под корпоративным VPN:** commit, push и pull в корпоративный GitLab без VPN не проходят.





Роли, навыки, правила, инструменты

Роль

исполнитель под вид работы: составитель ФТ, составитель БТ, составитель API

Навык

правила конкретного артефакта или источника: Use Case, описание UI, оцифровка документа

Правила

подключаются в каждую сессию

Инструменты

локальные программы разбора PDF, DOCX и XML — работают на Python, окружение ставится один раз по инструкции

Полный перечень — в навигаторе. В эфире его не читаем.



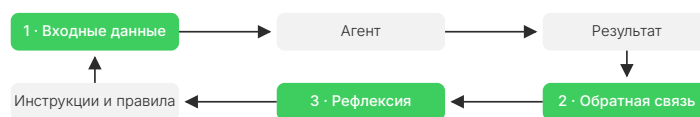


Одно слово в чат





Три фундамента работы с ИИ



Данные

Что не попало в контекст — того для модели нет. Источники — в проект, версия зафиксирована.

Обратная связь

Плох результат — правим не результат, а постановку или инструкцию. Запрет — через причину, не через «убери».

Рефлексия

Вечером `reflector` переводит день в правила. Следующая сессия стартует с накопленного.

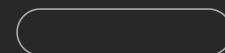
Правила и инструкции строим фундаментально. Модель можно сменить: Kilo Code переключает модель под задачу. Правила остаются.

Своя подписка — можно: к Kilo Code подключаются GPT и другие провайдеры. Claude — только по API, по подписке нельзя. Доступ и VPN для личной подписки — на вас.





2–3 вопроса из чата. Остальные — в блок 7.





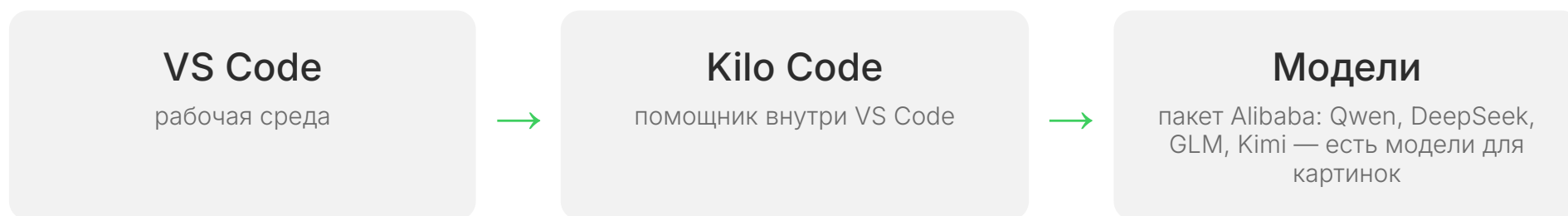
БЛОК 3

23–40 минут





Рабочий контур аналитика



Корпоративный GitLab — Золотой стандарт и ваш репозиторий под вашей фамилией; история работы и обратная связь. Внешние хостинги запрещены ИБ

Почему Kilo Code

Модель переключается под задачу, подключается любой провайдер. В Золотом стандарте подобраны температура и настройки, чтобы недорогие Flash-модели следовали инструкциям. Флагманы сильны, но дороже и чаще уходят в выдумки.





Золотой стандарт

Общий набор ролей, навыков, правил и инструментов. Одинаков у всех. Репозиторий `kilo-canon`, папка `.kilo`. Проектов в нём нет.

Его же называем **каноном**: вы им пользуетесь, правит владелец канона. «Подключить в корень» — папка, которую вы открыли в VS Code, становится копией `kilo-canon`.

Адрес: git.dos.softlab.ru/rnd/ai-analytics/kilo-canon

`projects/` — ваш репозиторий в GitLab под вашей фамилией: рефлексия и все ваши проекты

Золотой стандарт · `.kilo` — у себя не правим, изменения предлагаем владельцу

`agents`

`skills`

`rules`

`tools`

Обновления канона приходят сами: роль `devops` проверяет при старте сессии и подтягивает. Ваши проекты она не трогает. Если изменились правила — скажет начать новую сессию. Вручную — «проверь канон».





Три слоя памяти

Память проекта · `projects/<проект>/MEMORY.md` — как устроен именно этот проект. Читается по фразе «Работаем с <проект>»

Межпроектная рефлексия · `projects/reflection/` — опыт, который пригодится во всех ваших проектах. Пишет `reflector` в конце дня

Бэклог канона — дефект роли, навыка или инструмента `reflector` отправляет владельцу канона; исправление приезжает всем

Три слоя, чтобы не раздувать один файл и не грузить сессию тем, что ей не нужно: в проекте агент читает только память проекта.

Как пользоваться: агент ошибся → скажите ему, что не так → когда исправил — «Сделай рефлексию». По слоям он разложит сам; заглядывать в бэклог канона не нужно.





Рефлексия

Не контроль и не поиск чужих ошибок. Рефлексия копит реальный опыт команды: какие ошибки встречались и как их решали.

На этом опыте потом строится корпоративный ИИ, который понимает наш контекст.





Навигатор по ролям и навыкам

Полный навигатор — какие роли, какие навыки, с какой команды начинать — лежит внутри Золотого стандарта: файл `navigator.html` в корне рабочего места, рядом с `AGENTS.md`, открыть в браузере. Приезжает с обновлениями канона. Здесь — кого звать чаще всего.

БТ по регуляторике — проекты ДБО	<code>bt-writer</code>	«Работаем с <проект>»
Use Case, UI, API — проекты ДБО	<code>business-analyst</code>	«Работаем с <проект>»
ТЗ и ТЗ-Форматы — коннекторы	<code>tz-writer</code>	«Работаем с <проект>»
ФТ со смежными системами	<code>ft-writer</code>	«Работаем с <проект>»
Рефлексия в конце дня	<code>reflector</code>	«Рефлексия за сегодня»
Выгрузить работу в Git, обновить канон	<code>devops</code>	«Выгрузи работу по проекту»
Структура нового проекта	<code>layout-builder</code>	«Заведи раскладку проекта <имя>»

Навигатор пополняем по вашей обратной связи. Вопросы — в чат аналитиков в Telegram, топики: «ИИ · Настройка GitLab» · «ИИ: Золотой Стандарт / Канон» · «ИИ · Подключение проекта» · «ИИ · Общие вопросы».





Одним словом: Word / Confluence / Redmine / SVN / локальные папки / другое



Шаблон проекта

- Структуру руками не строим. Роль `layout-builder` заводит каталоги под тип работы: коннекторы или проекты ДБО и другие.
- Навыки обучены под эту структуру — поэтому её держимся.
- Новый проект начинаем с раскладки: проект у вас уже есть, но в рабочем месте его ещё нет — `layout-builder` закладывает основу.
- Сначала раскладка, потом роли-писатели — они берут адреса из паспорта раскладки `99-governance/layout.json`. Тип назовите сразу: «это коннектор» или «это ДБО».

Нужны ваши случаи

Работаете в проекте вместе с коллегами и держите в `projects/` и свой проект, и общий? Напишите нам: общий проект будем настраивать отдельным репозиторием — на ваших примерах.





Начинаем работу: проект + артефакт

Агенту называем **проект** и **артефакт**: `Работаем с <артефакт> по проекту GenBank-Digital-Ruble-DBOFL` и путь к файлу. Артефакта ещё нет — агент сохранит его по структуре проекта. Дальше агент сам, до первого ответа:

1 Общая память специалиста —
`projects/reflection/MEMORY.md`

2 Память проекта — `MEMORY.md` в папке проекта

3 `README.md` и паспорт раскладки
`99-governance/layout.json` —
отсюда все адреса

4 Проектные правила из паспорта раскладки: словарь, регламенты, реестр документов

5 Первой строкой говорит, что прочитал. Результаты кладёт только в каталоги из паспорта раскладки

Куда смотреть вам

`findings.md` — что подтверждено. `open-questions.md` — на что агент не смог ответить сам. Ваши ответы — обратная связь: из них складывается его опыт работы с вами.





МСР: живые корпоративные системы

Confluence

читает страницу по ссылке из задания и публикует статью в указанное пространство

Figma

читает макеты

Redmine

следующий на подключение

Работают с теми же моделями пакета Alibaba, под корпоративным VPN. Персональный токен — в `.env`, настраивается один раз.

Агент обращается к системе по ссылке из задания — выгружать страницы вручную не нужно.





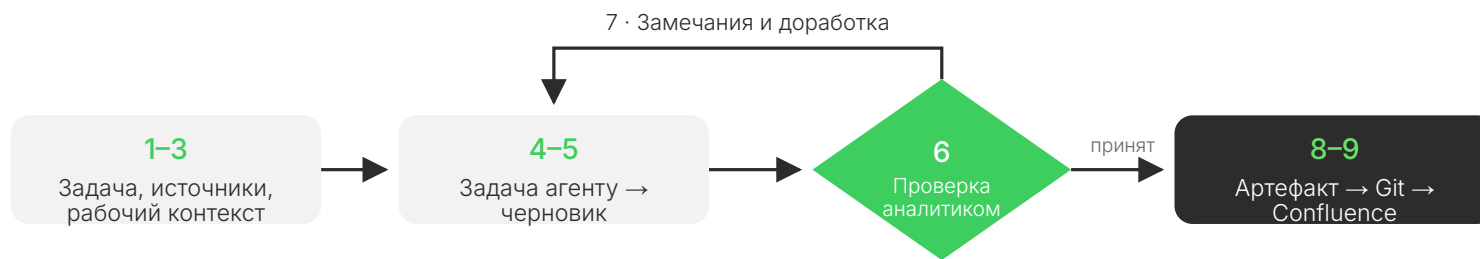
БЛОК 4

40–52 минуты





От задачи до коммита



1 Постановка задачи

2 Определение источников

3 Рабочий контекст: Золотой стандарт, МСР, источники

4 Постановка задачи агенту

5 Промежуточный результат

6 Проверка аналитиком

7 Уточнение и доработка

8 Финальный артефакт

9 Публикация в Space проекта в Confluence

10 Рефлексия — в конце дня

11 Коммит и отправка в Git

Шаги те же — меняется среда: источники собираются в одно рабочее пространство, **черновик готовит агент, принимает человек**.





Номер шага в чат





БЛОК 5.1 · КЕЙС

52–62 минуты





ДБО ФЛ ЦР ГенБанк: задача → инструмент → результат

Задача

Подготовить БТ по всем операциям ФЛ с цифровым рублём в ДБО ФЛ на основе актуальных требований Банка России, документации вендора и архитектуры проекта.

Роль / навык

Агент в Kilo Code сопоставляет источники и готовит черновики БТ по правилам проекта. Аналитик проверяет содержание и принимает решения.
Роли: `bt-writer` · `vision` для схем · `confluence` для публикации

Результат

17 БТ за 8 дней. Реестр БТ с трассировкой, подготовленные требования и открытые вопросы. Проверенные материалы переданы заказчику и опубликованы в Confluence.

Дальше — живой показ: проект в VS Code → артефакты в Confluence и документах.



Как это выглядит в работе

ДБО ФЛ ЦР ГенБанк, БТ по цифровому рублю — кто что делает

	Источники	Реестр	Инструкция на БТ	Черновик и вычитка	Утверждено
Алёна	собирает требования ЦБ, вендора, UI-стандарты, архитектурную схему	собирает реестр сценариев — он же таблица трассируемости	задаёт правила: цель, коллизии, входные данные, маппинг на шаблон	вычитывает, отвечает на вопросы в той же сессии	«переведи в статус утверждён»; публикует в Confluence под своей учёткой
Агент	PDF → MD; схему-картинку → описание в MD	дополняет реестр, держит ссылки на источники	пишет инструкцию на каждую БТ, готовит промт	пишет черновик в черновики/, ведёт открытые вопросы и трассировку; строит диаграммы PlantUML сам	переносит в согласованные, переименовывает, ставит ссылку в реестр

Роли: `bt-writer` пишет по инструкции на каждую БТ · `vision` читает схемы · `reflector` вечером правит инструкции



БЛОК 5.2 · КЕЙС

62–70 минут





ДБО ЮЛ Универсал Банк (Узбекистан): было → стало

Было

Плагин Claude Code. БФТ писала сама и отдавала агентам — по правилам они готовили use case и пользовательские сценарии. Результат — документы в Confluence. Каждое действие агента надо было подтверждать.

Стало

С этой недели — Kilo Code по Золотому стандарту. Агент работает, я параллельно занимаюсь своим, потом вычитываю и правлю.
Средняя БФТ — за два часа вместе с проверкой. Раньше на небольшую уходил день.

Сейчас в работе

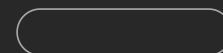
Валютные документы: описание UI по кейсам, API, документооборот — диаграммы активности по коду.
Роли: `business-analyst` · MCP
Confluence и Figma

На экране: БФТ и артефакты в Confluence.





Ответим в блоке 7





БЛОК 6

70–79 минут





Как начать у себя: три этапа

Настроить — один раз, до следующей встречи

VS Code + Kilo Code, ключ Alibaba — выдаёт Игорь, заявка через Алёну

Ключ SSH — агент создаёт, копируете кнопкой из ответа → GitLab: аватар → Preferences → слева Access

Клонировать Золотой стандарт из GitLab — он и есть корень; **заккрыть и открыть VS Code** — появятся роли

Окружение `.venv` и диаграммы — доустанавливает `devops`; с вас — Python 3.14

Свой репозиторий в `rnd/ai-analytics` — пустой, без README → `projects/`; подключает `devops` отдельно от канона: канон и ваши проекты друг друга не затирают

Токены MCP — в `.env`

Первый запуск `reflector` — аудит готовности рефлексии

Работать — каждая задача

Источники — в папку проекта

Проект и артефакт — «Работаем с <артефакт> по проекту <имя>»; агент говорит, что прочитал

Роль под артефакт — по навигатору

Проверка; плох результат — правим инструкцию

Вечером — «Рефлексия за сегодня»

Перед первой задачей — завести проект, один раз на проект: следующий слайд.

Грабли первого подключения: автоперевод браузера в GitLab — выключить; ошибка модели видна в строке состояния над полем ввода, а не в чате; `.doc` пересохранить в `.docx`. Превью PlantUML — расширение Markdown Preview Enhanced.

Пошагово со скриншотами — в навигаторе (`navigator.html` в Золотом стандарте), вкладка «Подключение к GitLab». Стартовая страница в Confluence: [wiki.softlab.ru · pageId=67998042](https://wiki.softlab.ru/pageId=67998042)



Завести проект — один раз на проект

1 Папка в `projects/` → `layout-builder` : «Сформируй в этом проекте структуру каталогов по канону» + тип: «это коннектор» / «это ДБО». Каркас — каталоги и паспорт раскладки `99-governance/layout.json`

2 Исходники — в `00-sources/` до первой задачи: архивы распаковать, `.doc` → `.docx` , «(1)» убрать; поменяли состав — `repo-keeper` проиндексирует

3 Содержание паспорта проекта — `business-analyst` на модели поумнее: «Начни с вопросов, ответы на которые помогут тебе сформировать паспорт проекта»

4 Шаблоны — в папку шаблонов проекта `98-templates/`

5 Роль под артефакт: «Работаем с <проект>. Задача: ...» + «Верни выводы о готовности начать выполнять текущую задачу» · смотреть `open-questions.md`

6 `devops` : «выгрузи» → обновить страницу проекта в GitLab

Почему в таком порядке

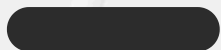
Работа над проектом начинается с паспорта — это фундамент: он даёт агенту ориентиры, и только потом — артефакты. Роли-писатели берут адреса из паспорта раскладки. Нет раскладки — результат некуда положить. Нет папки или `layout.json` — шаг 1 и тип; паспорт проекта пустой — шаг 3.





БЛОК 7

79–87 минут





Вслух или в чат



СПАСИБО

Что дальше

- Навигатор аналитика — внутри Золотого стандарта (`navigator.html`); стартовая страница — в Confluence
- Подключение к GitLab — по шпаргалке в навигаторе, за неделю; первая линия — Алёна
- Встреча по настройке — если в чате наберутся «да»
- Вопросы и обратная связь — чат аналитиков в Telegram, топики по ИИ
- Не хватает шага в навигаторе — напишите, чего не хватает: дополним